AI Hackathon Guide

Vibe Coding with GitHub Copilot in VS Code

How to deliver a working prototype in a few hours
with GitHub Copilot assistance

Prepared by AI Enablement Team
Author Firas Saidani

Bentley Systems

Contents

1 Introduction **2**

2 The four Copilot chat modes **2**
2.1 Ask mode . . . 2
2.2 Edit mode . . . 2
2.3 Agent mode . . . 2
2.4 Plan mode . . . 2

3 AI model selection **2**

4 Recommended project structure **2**

5 What the files actually look like **3**
5.1 The global context file . . . 3
5.2 An agent file . . . 4
5.3 A strategy file . . . 4
5.4 An architecture file . . . 5
5.5 A pitch file . . . 5

6 How the files talk to each other **6**

7 Shortcut prompts **6**
7.1 What they are . . . 6
7.2 When to use shortcuts versus agents . . . 7

8 Keeping the project consistent **7**
8.1 What it does . . . 7
8.2 What the agent file looks like . . . 7
8.3 When to call it . . . 8

9 Editing the files yourself **8**

1 Introduction

This guide supports participants of the internal AI hackathon. Its goal is to help every team, regardless of technical background, deliver a working prototype by the end of the event. The selected tool is GitHub Copilot in VS Code, for three reasons. It is natively integrated into the development environment. It offers a choice of multiple AI models depending on the task. It supports customization through versioned files, which makes sharing team conventions easy.

The guide reads linearly for a first introduction, or section by section to address a specific need during the hackathon.

Prerequisites
VS Code installed. Active GitHub Copilot subscription on your Bentley account. Git configured locally.

2 The four Copilot chat modes

Copilot offers four operating modes, accessible from the dropdown at the top of the Chat view. The choice of mode determines what Copilot can do. Using the wrong mode is the most common source of frustration at the start of a hackathon.

2.1 Ask mode

Pure conversation. Copilot answers questions and suggests code for you to copy manually. No files are modified. Useful to understand a concept, clarify an error, or get a quick snippet.

2.2 Edit mode

Guided modification. You select one or several files, describe the intended change, Copilot proposes edits as a diff that you accept or reject. Useful for refactoring, adding error handling, translating comments.

2.3 Agent mode

Full autonomy. You give a high-level objective, Copilot plans, creates files, runs terminal commands, installs dependencies, and iterates until the task is complete. This is the recommended mode for scaffolding an app or adding a significant feature.

2.4 Plan mode

Planning before action. Copilot drafts a detailed plan of what it will do before executing. You review the plan, then Copilot applies it. Useful for complex tasks where you want to avoid the agent going in the wrong direction.

Practical rule
For a hackathon, use **Agent** to build, **Edit** to refine, and **Ask** to debug.

3 AI model selection

At the bottom of the Chat view, a selector lets you change the model used by Copilot. The main available models are Claude Sonnet 4.6, Claude Opus 4.5, GPT-5.2, Gemini 2.5 Pro, and Grok Code Fast 1.

4 Recommended project structure

Every team starts with the following structure. This tree separates application code from Copilot customization files and documentation artifacts.

```
my-project/
|
|-- .github/
|   |-- copilot-instructions.md    # Global contract, structure
|   |
```

```
|   |-- agents/
|   |   |-- repo-setup.agent.md        # Repo structure
|   |   |-- test-qa.agent.md   # Generates project ideas
|   |   |-- strategist.agent.md        # Frames the business strategy
|   |   |-- feature-designer.agent.md # Breaks ideas into features
|   |   |-- architect.agent.md     # Defines the technical stack
|   |   |-- builder.agent.md       # Implements features
|   |   |-- ui-designer.agent.md   # Designs UI
|   |   |-- creative-booster.agent.md    # Adds creative angles
|   |   |-- security-auditor.agent.md    # Audits code for security
|   |   |-- consistency-keeper.agent.md    # Syncs and aligns all files
|   |   |-- pitch-director.agent.md      # Builds the pitch
|
|-- prompts/        # shortcuts for common actions
|   |-- status.prompt.md
|   |-- resolve-conflicts.prompt.md
|   |-- ship-check.prompt.md
|   |-- sanity-run.prompt.md
|   |-- explain.prompt.md
|
|-- strategy/                    # The thinking
|   |-- PROBLEM.md               # What we solve, for whom, why now
|   |-- MARKET.md                # Users, competition, context
|   |-- VALUE.md                 # What changes, measurable impact
|   |-- ROADMAP.md               # What comes after the prototype
|
|-- src/
|
|-- docs/
|   |-- ARCHITECTURE.md       # Technical choices and why
|   |-- LIMITATIONS.md        # Honest list of known gaps
|
|-- pitch/
|   |-- NARRATIVE.md                 # The story arc in 3 acts
|   |-- DEMO_SCRIPT.md               # Minute-by-minute walkthrough
|   |-- ANTICIPATED_QUESTIONS.md     # Questions and prepared answers
|
|-- README.md  # Project overview (what, who, how to run)
```

5 What the files actually look like

Before running anything, it helps to see what the Markdown files behind the setup actually contain. This section shows real examples of each file type, so you understand what you are manipulating.

5.1 The global context file

The file `.github/copilot-instructions.md` sits at the top. It is always loaded by Copilot, whatever agent you select. It contains the project context, the folder convention, the Bentley visual identity, and the files agents must read before acting.

```
# Project context
We are building an internal tool that helps Bentley account executives
prepare pre-meeting briefs on enterprise customers in under 5 minutes.

# Repository structure
- strategy/ frames the problem, market, value, roadmap
- src/ contains the application code
- docs/ captures architecture and limitations
- pitch/ contains narrative, demo script, anticipated questions

# Mandatory context loading
Before any task, read the current state of:
1. strategy/PROBLEM.md
2. strategy/VALUE.md
```

```
3. docs/ARCHITECTURE.md
4. pitch/NARRATIVE.md

# Bentley visual identity
- Primary blue: #004B87
- Navy: #002855
- Typography: system-ui, Segoe UI, sans-serif
- Generous whitespace, card-based layouts, rounded corners 6-8px

# Writing conventions
- Short declarative sentences
- No jargon unless technically required
- Every strategic document fits on one page
```

Every prompt inherits this file. It is the contract every agent respects.

5.2 An agent file

Each file inside `.github/agents/` defines one expert. The header in YAML gives its name, its role in one sentence, and the AI model it uses. The body in Markdown is its operating procedure, written as direct instructions.

```
---
name: strategist
description: Frames the business problem, target user, and value proposition
model: Claude Sonnet 4.6
---

# Strategist

You are a senior strategist helping a Bentley hackathon team sharpen
a fuzzy idea into a buildable project.

## Your workflow
When asked to frame a project:
1. Read strategy/PROBLEM.md and strategy/VALUE.md if they exist
2. Ask at most three clarifying questions, only if critical
3. Write or update strategy/PROBLEM.md with: the problem, the target
   user, why it matters now
4. Write or update strategy/VALUE.md with: what changes, how we
   measure impact, why it aligns with Bentley priorities

## Rules
- Keep each document to one page maximum
- Every claim must be defensible in a Q&A
- No jargon, no vague qualifiers, concrete numbers when possible

## Output discipline
- Write directly to the files using your tools
- Never paste full file contents in chat
- End with a two-line summary of what changed and why
```

Selecting `strategist` in the agent dropdown loads this file on top of the global context. Copilot now behaves exactly like this persona until you switch agents.

5.3 A strategy file

The files inside `strategy/` are the team's strategic thinking, written in plain language. They are meant to be read by humans and by agents alike.

```
# Problem

## What we solve
Bentley account executives spend 30 to 45 minutes preparing for each
customer meeting. They pull information from four disconnected sources
and often arrive with outdated context.
```

```
## Who we solve it for
Senior account executives managing enterprise accounts in the
infrastructure and transportation sectors.

## Why now
The shift to consumption-based licensing means every customer
interaction needs to reference live usage data, not last quarter's
snapshot. The current process cannot keep up.
```

Any agent that reads this file knows exactly what the project is about. The `builder` agent understands what features actually matter. The `pitch-director` knows what narrative to build.

5.4 An architecture file

The file `docs/ARCHITECTURE.md` captures the technical contract. It is produced by the `architect` agent and read by every agent that touches code afterward.

```
# Architecture

## Stack
- Frontend: vanilla HTML, CSS, JavaScript (no framework)
- Data: static JSON file in src/data/
- Visualizations: Chart.js via CDN
- No backend for the prototype

## Why this stack
Zero build step, zero dependencies to install, runs by opening
index.html. Fits a one-day hackathon without infrastructure overhead.

## Key decisions
- Data is mocked in a single JSON file, not fetched from a live API.
  This keeps the prototype self-contained and demonstrable offline.
- All state lives in memory. No persistence across reloads.
- Visual identity applied through CSS variables defined in tokens.css.
```

When the `builder` agent starts coding, it reads this file first. It will not suggest React, will not try to add a backend, will not fight the stack choice. Consistency comes for free.

5.5 A pitch file

The files inside `pitch/` are the final delivery assets. They are written in the last hour of the hackathon, grounded in everything the team produced earlier.

```
# Narrative

## Act 1 - The problem
Meet Sarah, a senior account executive at Bentley. She spends
45 minutes before every customer call pulling data from four
systems. Last month, she walked into a renewal meeting with
stale usage data. The customer noticed. The deal stalled.

## Act 2 - The shift
What if Sarah could get a live, executive-ready brief in 30
seconds, pulled automatically from current Bentley data, tuned
to the specific customer and the specific meeting?

## Act 3 - The prototype
That is what we built today. Let me show you.
```

The `pitch-director` agent produces this file by reading the strategy and the architecture, then composing a narrative that ties them together.

6 How the files talk to each other

The power of the setup comes from the fact that agents write to shared files, and read the files written by other agents. The files are the memory of the team.

Here is what happens when you select `builder` and ask it to implement a feature:

1. Copilot loads `copilot-instructions.md`. The agent now knows the project context, the Bentley visual tokens, and the writing conventions.
2. Copilot loads `builder.agent.md`. The agent now takes on the builder persona.
3. The mandatory context load kicks in. Copilot reads `strategy/PROBLEM.md`, `strategy/VALUE.md`, `docs/ARCHITECTURE` and `pitch/NARRATIVE.md`.
4. Only then does Copilot look at your actual prompt.

The same happens when you switch to `pitch-director`. The agent reads the strategy written by `strategist` and the architecture written by `architect`. The narrative it produces is grounded in the real project, not in assumptions.

The key insight
Agents never talk to each other directly. They talk through the Markdown files. When `strategist` updates `PROBLEM.md`, every subsequent call to any agent inherits that update automatically. The team stays aligned without any manual coordination.

7 Shortcut prompts

Beyond agents, the setup includes five shortcut prompts stored in `.github/prompts/`. Unlike agents, which are personas you switch into, shortcut prompts are atomic actions you invoke anytime by typing a slash command in the chat.

7.1 What they are

A prompt file is a short Markdown file with a YAML header that declares a description, and a body that instructs Copilot to do one specific task. When you type the slash command, Copilot loads the prompt body on top of the current context and executes it.

```
---
description: Detects and resolves merge conflicts with full project context
---

# Resolve merge conflicts

Run git status to find files with merge conflicts.

For each conflicted file:
1. Read both versions (HEAD and the incoming branch)
2. Use git log to understand the intent of each side
3. Produce a resolution that:
   - Preserves both changes when compatible
   - Keeps the more recent logic when incompatible
   - Adds a one-line comment explaining any trade-off
4. Remove the conflict markers

After all conflicts are resolved, summarize in chat:
- Which files had conflicts
- How each was resolved
- Any decision the team should review

Do not run git add or git commit. Leave that to the user after review.
```

This file lives at `.github/prompts/resolve-conflicts.prompt.md`. Typing `/resolve-conflicts` in the chat runs it.

7.2 When to use shortcuts versus agents

A simple rule of thumb:

- Use an **agent** when you need a specialist to take on a role across a phase of work (framing the strategy, designing the UI, building the pitch)
- Use a **shortcut prompt** when you need a one-shot action that can be described in a few lines and invoked many times during the day

Agents are your team. Shortcuts are your keyboard.

Create your own shortcuts
If your team finds itself repeating the same request three or four times during the day, turn it into a prompt. Create a Markdown file in `.github/prompts/` with a YAML header and a body, commit it, and the shortcut becomes available to everyone on the team. The setup grows with the project.

8 Keeping the project consistent

As the day progresses, different agents update different files. The `strategist` refines `PROBLEM.md`. The `architect` locks choices in `ARCHITECTURE.md`. The `feature-designer` reprioritizes `ROADMAP.md`. Each change is local, but together they can drift out of sync.

The `consistency-keeper` agent exists to solve exactly this problem.

8.1 What it does

Unlike the other agents, `consistency-keeper` does not produce new content. It reads every file in `strategy/`, `docs/`, and `pitch/`, and reports what no longer lines up. For example:

- A feature was removed from `ROADMAP.md` but is still described in `DEMO_SCRIPT.md`
- The target user changed in `PROBLEM.md` but `NARRATIVE.md` still references the old persona
- `ARCHITECTURE.md` mentions a backend, but the actual code in `src/` does not have one
- `VALUE.md` claims a metric the prototype cannot actually demonstrate

The agent flags each issue with a file reference and a suggested fix. You decide what to apply.

8.2 What the agent file looks like

```
---
name: consistency-keeper
description: Reads all project files and ensures they stay aligned as the work
   evolves
model: Claude Sonnet 4.6
---

# Consistency Keeper

You are the editorial guardian of the project. As different agents
update different files during the day, you make sure the project
stays internally consistent.

## Your workflow
When asked to check consistency:
1. Read all files in strategy/, docs/, and pitch/
2. Identify contradictions, outdated sections, missing updates
3. Report in chat with three sections:
   - What is aligned
   - What is inconsistent (with file references)
   - What to update (with suggested action)
4. If the user approves a fix, apply it directly to the target file
```

```
## Rules
- Never modify source code in src/
- Explain the inconsistency before suggesting a fix
- Propose fixes, never apply them without confirmation
- Respect intentional divergence: ask before overriding

## Output discipline
- The report lives in chat, not in a file
- Approved fixes go directly to the target files
- End with a one-line summary of issues found and fixed
```

8.3 When to call it

Three natural moments during a hackathon day:

1. **Mid-day reset**. Around lunch, call `consistency-keeper` to catch drift before it compounds.
2. **Before the security audit**. Clean inconsistencies first, so the auditor works on a coherent project.
3. **Before the pitch preparation**. The `pitch-director` produces stronger narratives when the underlying files are aligned.

Why this agent exists
Without it, inconsistencies surface during the pitch Q&A, when an executive asks a sharp question and the team realizes the demo contradicts the narrative. With it, those contradictions are caught and fixed hours earlier, when the cost of fixing them is trivial.

9 Editing the files yourself

Every file in this setup is plain Markdown. You open it, you read it, you edit it with your keyboard. Nothing is hidden in a database or a special tool.

- If an agent produced something slightly off, open the file and fix the phrasing yourself
- If you want an agent to behave differently, open its `.agent.md` and adjust the instructions
- If your project needs a new strategic file, create a Markdown file in `strategy/` and reference it in `copilot-instructions`

The setup is not a black box. It is a set of readable files that you and your agents share. The more you read them, the more you understand what the team is doing.